

📘 04차시 LoRA 원리와 PEFT 라이브러리 기초

교육시간: 1일차 14:00–15:00 / 60분 | **문서 기준일:** 2026-11-04

01 | 개요·시간 배분

학습 주제: LoRA(Low-Rank Adaptation, 저랭크 적응)와 PEFT(Parameter-Efficient Fine-Tuning, 매개변수 효율적 미세조정). 교육시간: 1일차 14:00~15:00 (60분). 0~10분: 전체 미세조정의 비용 / 10~25분: LoRA 행렬 원리 / 25~35분: 코드 분석 / 35~50분: 실행 / 50~60분: 검증과 평가. 학습 결과: 원본 가중치 동결, LoRA 추가 행렬, 학습 가능 매개변수 비율을 직접 확인한다. 주의: 본 차시는 학습 완료가 아니라 '학습 준비 구조와 매개변수 검증'을 목표로 한다.

---

02 | 선수지식과 문제 정의

02차시: 토큰·Transformer·자기회귀 생성 구조를 이해한다. 03차시: 모델 다운로드와 추론을 수행한다. 전체 미세조정은 대형 가중치 전체의 기울기와 옵티마이저 상태를 보관하므로 메모리 부담이 크다. 예시: 5억 개의 매개변수를 FP32로 보관하면 가중치 자체만 약 2GB(십진 GB)이며, 학습 시 활성값·기울기·옵티마이저 등 추가 공간이 필요하다. LoRA는 원본 W를 고정하고, 작은 추가 행렬 A·B만 학습하여 필요한 학습 매개변수를 줄인다. 단, 메모리 절약이 곧바로 추론 속도 향상이나 학습 품질 보장을 뜻하지 않는다.

03 | LoRA 수학과 계산

선형 계층: y = Wx. 원본 W의 크기를 d_out × d_in이라 하자. LoRA 업데이트: ΔW = BA, B의 크기는 d_out × r, A의 크기는 r × d_in. 실제 출력: y = Wx + (α/r)BAx. r은 rank(랭크), α는 scaling(스케일링) 계수이다. 전체 학습 파라미터 수: d_out × d_in. LoRA 추가 파라미터 수: r × (d_in + d_out). 예시 d_in=d_out=1024, r=8이면 전체 1,048,576개에 비해 LoRA는 16,384개(1.5625%). 이 비율은 단일 선형층의 이론 예시이며 모델 전체의 실제 비율과 다르다.

04 | 구조도와 데이터 흐름

입력 x → [원본 W : 동결] → 원본 출력 Wx ─┐ └→ [A : 학습] → [B : 학습] → (α/r)BAx ─┤ → 더하기 → 출력 y 역전파: 손실 → A·B에만 기울기 반영, 원본 W는 변경하지 않는다. 모델 구조: Transformer Block → Attention → q_proj / v_proj → LoRA 모듈 삽입. q_proj는 Query(쿼리) 투영, v_proj는 Value(밸류) 투영이다. 모델마다 계층 이름이 다르므로 target_modules를 임의로 복사하지 말고 실제 모듈명을 확인한다. 주의: LoRA가 모델 전체에 자동으로 적용되는 것이 아니라 지정된 계층에만 삽입된다.

05 | Ubuntu 실습 준비

터미널에서 작업 디렉터리 생성: mkdir -p ~/lab_llm/ch04 && cd ~/lab_llm/ch04 기존 가상환경 활성화(예): source ~/lab_llm/.venv/bin/activate Python 확인: python --version → 3.11.x 예상 패키지 확인: python -m pip show torch transformers peft 설치 예: python -m pip install "transformers>=4.45,<5" "peft>=0.13" torch 인터넷 연결과 모델 저장 공간이 필요하며, 최초 실행 시 모델 파일이 다운로드된다. GPU 확인(선택): nvidia-smi. CPU에서도 구조 확인은 가능하지만 다운로드·로딩에 시간이 걸린다. 설치 버전은 실행 전 고정 기록하고 호환성 문제가 있으면 함께 조정한다.

06 | 전체 코드와 핵심 설정

실습 파일: lesson04_peft_inspect.py (ZIP에 포함된 실행 가능 소스). MODEL_ID = "Qwen/Qwen2.5-0.5B-Instruct" # 교재용 모델 선택 config = LoraConfig(r=8, lora_alpha=16, lora_dropout=0.05, target_modules=["q_proj", "v_proj"], bias="none", task_type=TaskType.CAUSAL_LM) peft_model = get_peft_model(model, config) peft_model.print_trainable_parameters() 이 차시는 학습 루프를 수행하지 않는다. 09차시에서 SFT와 결합한다. 메모리 제약이 큰 장비에서는 모델 로딩이 실패할 수 있으므로 여유 메모리를 확인한다.

07 | 코드 해설·검증

AutoModelForCausalLM: 다음 토큰 예측용 인과 언어 모델을 불러온다. AutoTokenizer: 문자열과 토큰 ID 사이 변환기를 불러온다. LoraConfig: r=8은 추가 행렬의 내부 차원, lora_alpha=16은 크기 조절 값, lora_dropout=0.05는 학습 중 드롭아웃 확률이다. get_peft_model: 지정한 투영층에 학습 가능한 LoRA 모듈을 삽입한다. requires_grad=True인 매개변수만 trainable에 포함된다. 출력 예시: trainable=... total=... ratio=...% (숫자는 설치 버전·대상 모듈에 따라 달라짐). 검증 기준: 0 < trainable < total이며, 출력 목록에서 lora_A/lora_B가 발견되어야 한다.

08 | 따라하기 실습

① lesson04_peft_inspect.py를 ~/lab_llm/ch04/에 복사한다. ② python -m py_compile lesson04_peft_inspect.py 로 문법 확인. ③ python lesson04_peft_inspect.py 실행. 최초 다운로드는 시간이 걸릴 수 있다. ④ 학습 가능 매개변수 수, 전체 매개변수 수, 비율을 기록한다. ⑤ r=8을 r=4로 바꾼 뒤 다시 실행하고 두 결과를 비교한다. ⑥ r을 절반으로 줄이면 LoRA 계층 매개변수는 대체로 절반이 되지만 전체 비율은 원본 가중치 때문에 단순히 절반이 아닐 수 있다. ⑦ r=8로 되돌리고 실습 결과를 기록한다.

09 | 오류 대응과 확장

ModuleNotFoundError: 가상환경 활성화 및 python -m pip show peft 확인. OSError/HTTP 401: 모델 ID·접근 권한·네트워크 상태 확인. Out of memory: 다른 프로세스 종료, 메모리 여유 확보, CPU 사용 또는 더 작은 모델 검토. Target modules not found: 모델의 named_modules()로 q_proj/v_proj 존재 여부 확인. 라이브러리 API 변경: 설치된 버전과 공식 문서를 대조한다. 확장: r=4/8/16에 대해 trainable 비율 표를 작성하고 alpha/r의 의미를 설명한다. 실행 예시와 실제 측정값은 구분하고, 측정 장비·패키지 버전을 기록한다.

10 | 평가·체크리스트·참고

평가 1: LoRA에서 원본 W가 동결되는 이유를 설명한다. 평가 2: d_in=512, d_out=1024, r=8일 때 추가 파라미터 수를 계산한다. 정답: 12,288개. 평가 3: target_modules에 존재하지 않는 이름을 넣으면 어떤 오류가 발생하는지 설명한다. 평가 4: 학습 가능 비율이 낮아도 품질이 자동 보장되지 않는 이유를 설명한다. 체크: Python 3.11 / 모델 로딩 / LoRA 적용 / trainable 비율 기록 / 오류 로그 보존. 공식 참고: https://huggingface.co/docs/peft/main/en/conceptual_guides/lora 공식 참고: https://huggingface.co/docs/peft/main/en/package_reference/lora 공식 참고: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct 자료 작성일: 2026-11-04. 링크는 공식 문서 주소이며 실시간 API 교차 검증은 별도 필요. 다음 차시: 05 양자화·bitsandbytes — 가중치 저장 비트 수와 QLoRA의 연결.

부록 | 실습 코드

```
# lesson04_peft_inspect.py — 04차시 LoRA 적용 및 학습 매개변수 점검
# 설치: python -m pip install "transformers>=4.45,<5" "peft>=0.13" torch
# 실행: python lesson04_peft_inspect.py
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, TaskType, get_peft_model

MODEL_ID = "Qwen/Qwen2.5-0.5B-Instruct"  # 교육용 선택 모델

# 학습 가능한 LoRA 가중치를 확인하기 위한 CPU/GPU 공통 예제
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, dtype=torch.float32, low_cpu_mem_usage=False
)
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)

# Qwen2.5 계열 attention 투영층에 저랭크 행렬을 삽입
config = LoraConfig(
    r=8, lora_alpha=16, lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],
    bias="none", task_type=TaskType.CAUSAL_LM,
)
peft_model = get_peft_model(model, config)
peft_model.print_trainable_parameters()

# 실제 이름과 requires_grad로 학습 대상 확인
trainable = sum(p.numel() for p in peft_model.parameters() if p.requires_grad)
total = sum(p.numel() for p in peft_model.parameters())
print(f"trainable={trainable:,} total={total:,} ratio={100*trainable/total:.4f}%")
print("tokenizer vocab size:", len(tokenizer))
```

적대적 검증·개선 기록 (2026-11-05)

반영: 기존 코드·교안의 핵심 구조를 보존했습니다. 실제 GPU 실험은 별도 실행 검증이 필요합니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)